血糖调控智能体：基于模型学习与 Q-Learning (Assignment - 3)

> 对应课程内容：第 8 章 马尔可夫决策过程；第 9 章 无模型强化学习
> 发布时间：2026-10-26 ｜ 截止时间：2026-11-08

作业简介

本次作业要求每位同学**独立实现两种无模型强化学习路线**，并在课程提供的统一血糖调控环境上对比它们的差异：

1. **基于模型的学习（Part A）**：转移与奖励模型**未知**。你先从交互经验中学习模型（估计 $\hat{P}(s'|s,a)$ 与 $\hat{R}(s,a)$，对应第 9 章"从经验中学习模型"），再在学到的模型上运行价值迭代求策略（第 8 章：Bellman 最优方程、策略提取）。这是"先学模型、再规划"的两步路线。
2. **Q-Learning（Part B）**：模型同样未知，但**不学模型**。你直接从给定经验中用表格 Q-Learning 更新 $Q(s,a)$（第 9 章：Q-Learning 更新规则、异策略）。这是"绕开模型、直接学价值"的一步路线。主任务为**离线批量学习**：`learn()` 只需按给定顺序遍历 episodes 并逐条更新，不要在其中采样动作；ε-greedy 探索只用于报告的附加在线实验。

- 两条路线使用**同一份交互数据预算**，因此实验可以公平对比"学模型再规划"与"直接学 Q"的样本效率与最终性能。
- 课程提供统一的**血糖模拟器**与**接口检查程序**，所有人使用相同环境，结果可复现、可比较；正式评分程序由课程方保留。
- 你需要自己实现模型估计、价值迭代与 Q-Learning 的核心逻辑；**不得调用任何现成强化学习或规划库**。

一、任务概览

部分	对应章节	任务	提交文件
Part A	第 8+9 章	从经验中学习 $\hat{P}/\hat{R}$ + 价值迭代	`model_based.py`
Part B	第 9 章	Q-Learning + ε-greedy	`q_learning.py`

二、课程提供的环境与数据

课程随本讲义发布 `glucose_env.py` 与 `sanity_check.py`（两个文件与本讲义一起下载，无目录结构）。这两个文件**只读，不得修改**，本地测试时请放在 `assignment3/` 同级目录。正式评分程序（含参考答案与阈值）由课程方保留，不随讲义发布。

2.1 血糖模拟器 `glucose_env.py`

两个部分都只通过模拟器交互获得经验，**不能读取**任何真实转移表或奖励表。模拟器接口：

```
class GlucoseEnv:
    def __init__(self, seed=None): ...

    def reset(self) -> str:
        """重置环境，返回初始离散状态（'B1'–'B9'）。初始血糖 ~ N(6.5, 1.5)。"""

    def step(self, action) -> tuple:
        """执行动作，返回 (next_state, reward, done, info)。"""

    @property
    def actions(self) -> list:
        """返回动作列表 ['w', 'g', 'i1', 'i2']。"""
```

2.2 状态、动作与奖励

离散状态（共 9 个，按真实血糖所在区间划分）：

状态	血糖范围 (mmol/L)	含义	进入奖励 $R(s')$
B1	$[2, 4)$	严重低血糖（**终止**）	-10
B2	$[4, 6)$	轻度低血糖	-1
B3	$[6, 8)$	目标区间	$+1$
B4	$[8, 10)$	略高	0
B5	$[10, 12)$	轻度高血糖	-1
B6	$[12, 14)$	中度高血糖	-2
B7	$[14, 16)$	较高高血糖	-4
B8	$[16, 18)$	高血糖	-6
B9	$[18, 20]$	严重高血糖	-8

动作与成本（成本加到进入奖励上，即 $r = R(s') + c(a)$）：

动作	含义	成本 $c(a)$
`w`	维持观察	0
`g`	补糖	-0.5
`i1`	小剂量胰岛素	-0.2
`i2`	大剂量胰岛素	-0.4

- 时间步长 15 分钟，每条轨迹最多 $T = 200$ 步；

- 进入 B1 立即终止回合（类似悬崖：安全性约束）。注意：若 `reset()` 返回 B1，智能体仍需先选择动作；只有动作后转移到 B1 时才判定终止；
- 转移含随机性（进餐事件与代谢噪声），折扣因子 $\gamma = 0.9$，与课堂一致。

2.3 统一数据预算

为保证两条路线公平对比，评测程序用**均匀随机策略**在模拟器中采集固定数据集：

- **2000 条 episode**，每条最多 200 步；
- 每条经验为 $(s, a, r, s', \text{done})$ 五元组；
- Part A 与 Part B 使用**同一份数据**（不同题目评测时数据相同，随机种子固定）。

> **运行目录**：评测程序固定读取 `assignment3/model_based.py` 与 `assignment3/q_learning.py`，请在**包含 `assignment3/` 目录的上一级目录**运行所有评测/检查命令。

2.4 本地接口检查 `sanity_check.py`

```
python sanity_check.py        # 检查文件名、类名、接口与基本行为
```

`sanity_check.py` **不含任何参考答案或评分阈值**：通过仅代表评测程序能正确加载并调用你的代码，不代表实现得分。正式评分由助教在内部评分程序上完成。

三、统一实现协议

3.1 基于模型的学习 `model_based.py`

每位同学提交一个名为 `model_based.py` 的文件，其中必须包含一个名为 `ModelBasedAgent` 的类：

```
class ModelBasedAgent:
    """Model-Based RL Protocol v1 —— 课程评测程序将按此接口加载并调用你的智能体。"""

    def __init__(self, states, actions, gamma: float = 0.9, seed=None):
        """states 为状态名列表，actions 为动作列表。"""

    def learn_model(self, episodes: list):
        """
        从经验中估计模型。
        :param episodes: 列表的列表；每个 episode 是若干
                         (state, action, reward, next_state, done) 五元组。
        估计：
          P̂(s'|s,a) = N(s,a,s') / N(s,a)    （状态转移计数归一化）
          R̂(s,a)   = 该 (s,a) 上所有 reward 的样本均值
        对从未访问过的 (s,a)：P̂ 均匀分布，R̂ = 0，并在报告中说明这样处理的原因。
        """

    def run_value_iteration(self, theta: float = 1e-4, max_iterations: int = 1000):
        """在学到的 P̂/R̂ 上执行同步扫描的价值迭代，直到收敛或达到 max_iterations。"""

    def get_transition(self, state, action) -> dict:
        """返回你估计的 {next_state: prob}。"""

    def get_reward(self, state, action) -> float:
        """返回你估计的 R̂(s,a)。"""

    def get_value(self, state) -> float: ...
    def get_policy(self, state):
        """返回 argmax_a Q̂(s,a)；平局时取课程约定顺序中的第一个动作。"""
```

实现要求：

- 模型估计：对每个 (s,a) 用计数法估计 $\hat{P}$、用样本均值估计 $\hat{R}$；必须自己实现，不得使用现成统计库的高级接口；
- 价值迭代：使用第 8 章 Bellman 最优方程，在 $\hat{P}/\hat{R}$ 上做**同步**扫描；
- 策略提取：$Q(s,a)=\sum_{s'}\hat{P}(s'|s,a)\,[\hat{R}(s,a)+\gamma\hat{V}(s')]$，$\pi(s)=\arg\max_a Q(s,a)$。

3.2 Q-Learning `q_learning.py`

```
class QLearningAgent:
    """Q-Learning Protocol v1 —— 课程评测程序将按此接口加载并调用你的智能体。"""

    def __init__(self, states, actions, gamma: float = 0.9, alpha: float = 0.5,
                 epsilon: float = 0.3, epsilon_decay: float = 0.995,
                 epsilon_min: float = 0.05, seed=None):
        """epsilon 相关参数仅用于报告的附加在线探索实验；离线主任务不使用。"""

    def learn(self, episodes: list, mode: str = "fixed"):
        """
        从统一数据预算的 episodes 中按顺序做表格 Q-Learning 更新：
        逐条 episode、逐条转移调用 self.update()，不做任何动作采样。
        mode 参数保留接口一致性，本作业离线主任务中不使用探索率。
        """

    def update(self, state, action, reward, next_state, done: bool):
        """单步表格 Q-Learning 更新；done=True 时 target 中不含未来项。"""

    def get_q_value(self, state, action) -> float: ...
    def get_policy(self, state):
        """返回当前贪心动作（Q 表每行 argmax）。"""
```

实现要求：

- $Q(s,a)$ 初始化为 0；更新式与第 9 章一致：$Q(s,a) \leftarrow (1-\alpha)Q(s,a) + \alpha\left[r + \gamma \max_{a'} Q(s',a')\right]$；
- **不得读取**真实转移表或奖励表——只能使用给定的 (s,a,r,s') 经验（无模型）；
- 本作业的 Q-Learning 为**离线批量**设定（统一数据预算），探索对比放在报告的附加实验中（见 4.2）。

四、实现要求

4.1 Part A：基于模型的学习 + 价值迭代

- 对统一数据预算（2000 条 episode）统计 $\hat{P}/\hat{R}$，报告：
  - 每个 (s,a) 的访问次数分布（直方图），指出哪些 (s,a) 数据稀疏；
  - 学到的策略表（每个状态的 $\pi(s)$）与 $\hat{V}(s)$；
- 将学到的策略放入模拟器执行 **500 条**评估轨迹，报告：平均回报（**无折扣累计回报**，直接累加每步 r，不乘 γ）、处于目标区间 B3 的时间比例、进入 B1 的终止率；
- 数据量消融：分别用前 500 / 1000 / 2000 条 episode 学习模型并求策略，对比三者评估性能，讨论样本量对模型估计的影响；

- 在报告中解释至少一个状态上的策略选择逻辑（例如 B4 为什么选择 i_1 而不是 i_2，或解释 B8 上两条路线为什么可能不一致）。

4.2 Part B：Q-Learning

- 用统一数据预算按顺序更新 Q 表，报告最终 Q 表与贪心策略表；
- 将贪心策略放入模拟器执行 **500 条**评估轨迹，报告与 Part A 相同的三个指标（平均回报为无折扣累计回报）；
- 附加实验（报告要求，不计入接口检查）：自行在线采集数据训练 Q-Learning，对比固定 $\varepsilon = 0.3$ 与衰减 ε（$\varepsilon_0 = 0.3$，$\text{decay} = 0.995$，$\varepsilon_{\min} = 0.05$）的学习曲线（5 个随机种子，滑动窗口 50），联系第 9 章"探索的代价（Regret）"解释差异；
- 对比分析：
  - Part A 与 Part B 学到的策略在每个非终止状态上的一致率；
  - 相同数据预算下，两条路线的评估性能差异，并解释原因（模型估计误差 vs 直接价值估计）；
  - 讨论第 9 章收敛三条件（充分访问、学习率、$\gamma < 1$ 且奖励有界）在本任务中的体现。

五、评分标准

本作业最鼓励大家做好的是：**算法实现正确、实验完整、解释清楚**。评测重点不是追求最高回报，而是看你能否理解"学模型再规划"与"直接学 Q"两条路线的差异。

项目	分值
Part A 模型估计 + 价值迭代实现正确（评分程序 q1）	35
Part B Q-Learning 实现正确（评分程序 q2）	35
实验评估完整（指标、曲线、对比、消融）	15
报告清晰、分析到位	15

- 未通过内部评分程序对应题目的实现，该项实验分数最多得一半；
- 报告缺少要求的图表或对比实验将相应扣分。

六、报告要求

报告以 Markdown 撰写，保存为 `assignment3/report.md`（可另附 `report.pdf`）。内容包括：

1. **总体设计**：两条路线的关系（学模型再规划 vs 直接学 Q），你如何组织代码与实验。
2. **Part A 说明**：计数法估计 $\hat{P}/\hat{R}$ 的实现、未访问 (s,a) 的处理、价值迭代实现、策略表与价值表、数据量消融结果、至少一个状态的策略逻辑解释。
3. **Part B 说明**：更新式实现细节、最终 Q 表、与 Part A 的策略一致率与性能对比、附加探索实验的结果与解释、收敛三条件的讨论。
4. **实验评估**：所有要求的指标与图表；每个图表都要有简短的观察与解释。
5. **讨论与反思**：遇到的最大困难、两条路线的适用边界（何时学模型更划算、何时直接学 Q 更稳）、若继续改进会做什么（例如函数逼近，对应第 10 章预告）。

七、提交内容与方式

本学期全部通过你与教师共享的 **GitHub 课程仓库** 提交。在你的仓库中建立 `assignment3/` 目录：

```
assignment3/
├── model_based.py       # Part A 统一协议智能体
├── q_learning.py        # Part B 统一协议智能体
├── analysis.md          # 简答（见下）
├── main.py              # 实验入口：复现报告中全部图表
├── requirements.txt
├── figures/             # 报告引用的图表
└── report.md            # 实验报告（Markdown，可另附 report.pdf）
```

`analysis.md` 中回答以下两个问题（每题 3–5 句，中英文均可）：

1. 在本作业的数据预算下，"先学模型再规划"与"直接学 Q"哪条路线对稀疏访问的 (s, a) 更敏感？为什么？
2. 找出一个两条路线策略不一致的状态，并给出一个可能原因（模型估计误差 / 采样不足 / 价值传播路径不同）。

- 提供的 `glucose_env.py`、`sanity_check.py` **不要提交**（评测程序自带）。
- 截止判定：以截止时间前仓库中 `assignment3/` 目录的最后一次 push 为准；截止后不再接受补交修改。
- 请勿把数据文件、模型权重或无关大文件提交进仓库。

八、学术诚信

- 本作业须独立完成；与他人讨论思路可以，但代码不得共享或抄袭。查重工具将检查所有提交的算法文件。
- **禁止**调用任何现成强化学习 / 规划库（如 gymnasium、stable-baselines3 等）；允许 Python 标准库、numpy、matplotlib。
- **禁止**直接复制 Berkeley CS188 或其他公开课程的本作业对应代码。
- 引用任何外部资料（含教材、博客、开源代码片段）须在报告中注明出处。
- 违反学术诚信者按学校相关规定处理。

九、时间安排

节点	时间
作业发布	2026-10-26
提交截止	2026-11-08

建议节奏：第 1 周完成 Part A（模型估计 + 价值迭代）；第 2 周完成 Part B、实验与报告。